Republic of the Philippines
Laguna State Polytechnic University
Province of Laguna

COLLEGE OF COMPUTER STUDIES

Student Management System

DOCUMENTATION

Cuala, Christian Mer O.

Rebong, Julian Carl D.

Salazar, Marciano G.

Sombrano, Mrincess Allisha C.

Tagayun, Jannell J.

By: BSCS 2B - Group 2

Submitted to: Ms. Roxanne Garbo

TABLE OF CONTENTS

Introduction

The **ClassTrack** application is an innovative solution designed to assist educators and professors in managing various aspects of their academic responsibilities more efficiently. As educational systems become increasingly complex, the need for a well-organized and streamlined approach to managing class schedules, attendance, grades, and student performance has grown more critical. **ClassTrack** aims to address this challenge by providing a user-friendly platform that simplifies the administrative tasks typically faced by instructors, helping them save time and focus more on teaching. With features tailored to meet the specific needs of educators, **ClassTrack** enhances the management of academic activities, from tracking attendance to analyzing student performance, all in one centralized system.

Objectives

The primary purpose of **ClassTrack** is to facilitate the day-to-day operations within educational institutions, offering a platform where professors can efficiently manage their classes, track student progress, and access important resources. While students indirectly benefit from the system, it is specifically designed for instructors to simplify administrative tasks. Professors have the ability to manage class schedules, track attendance, monitor academic performance, and access relevant educational resources, all in one convenient interface.

In today's fast-paced educational environment, it is essential for instructors to quickly and efficiently access key information. **ClassTrack** provides a dashboard that allows

seamless navigation between various sections, such as class management, attendance, schedules, and resources. This organized structure helps professors save valuable time and focus more on teaching. Built on the **PyQt5** framework, **ClassTrack** offers a modern and responsive graphical user interface (GUI), ensuring ease of use with minimal training. It also integrates powerful features like charts for visualizing student performance, enabling professors to easily assess class trends and take timely actions based on the data.

The primary objectives of the Student Management System are as follows:

- **Efficient Data Management**: Enable the easy creation, modification, deletion, and retrieval of student records, streamlining the administrative process and eliminating paperwork.

- **Enhanced Accuracy and Reliability**: Minimize errors by storing data in a centralized, digital format, ensuring data consistency and reducing administrative workload.

- **User-Friendliness**: Provide a simple, intuitive interface that can be used by non-technical users to perform common tasks, such as adding students, updating records, and searching for information.

- **Improved Efficiency**: Automate student record management to save time, reduce administrative burden, and increase productivity within educational institutions.

- **Scalability**: Provide a system that can be expanded or modified to accommodate growing student populations and additional features as needed.

Pseudocode:

- **Main Program Flow**

```
BEGIN
  CALL InitializeDatabase()
  CALL DisplayLoginScreen()
END
```

- **Login Process**

```
FUNCTION DisplayLoginScreen()
  DISPLAY "ClassTrack Login"
  INPUT username
  INPUT password

  IF ValidateUser(username, password) THEN
    CALL LoadDashboard(username)
  ELSE
    DISPLAY "Invalid credentials. Please try again."
    CALL DisplayLoginScreen()
  END IF
END FUNCTION
```

- **Validate User Credentials**

```
FUNCTION ValidateUser(username, password)
  query = "SELECT * FROM users WHERE username
= '" + username + "' AND password = '" + password +
"'"

  result = ExecuteQuery(query)

  IF result IS NOT EMPTY THEN
    UPDATE users SET last_login =
CURRENT_TIMESTAMP WHERE username =
username
    RETURN TRUE
  ELSE
    RETURN FALSE
  END IF
END FUNCTION
```

- **Dashboard**

```
FUNCTION LoadDashboard(username)
  user_data = GetUserData(username)
  DISPLAY "Welcome, " + user_data.username

  DISPLAY Menu Options:
    1. Classes
    2. Attendance
    3. Grades
    4. Schedule
    5. Resources
    6. Logout
  INPUT choice
  SWITCH choice
    CASE 1: CALL ManageClasses()
    CASE 2: CALL ManageAttendance()
    CASE 3: CALL ManageGrades()
    CASE 4: CALL ManageSchedule()
    CASE 5: CALL ManageResources()
    CASE 6: CALL Logout()
    DEFAULT: DISPLAY "Invalid choice"
  END SWITCH
END FUNCTION
```

- **My Class Management**

```
FUNCTION ManageClasses()
  DISPLAY "Class Management"
  DISPLAY Options:
    1. View All Classes
    2. Add New Class
    3. Edit Class
    4. Delete Class
    5. Back to Dashboard
  INPUT action
  SWITCH action
    CASE 1: CALL ViewClasses()
    CASE 2: CALL AddClass()
    CASE 3: CALL EditClass()
    CASE 4: CALL DeleteClass()
    CASE 5: CALL LoadDashboard()
  END SWITCH
END FUNCTION
```

```
FUNCTION ViewClasses()
  query = "SELECT s.*, u.username FROM sections s
JOIN users u ON s.user_id = u.user_id"
  classes = ExecuteQuery(query)
  FOR EACH class IN classes DO
    DISPLAY class.section_name, class.subject,
class.room, class.section
  END FOR

    CALL ManageClasses()
  END FUNCTION
```

```
FUNCTION AddClass()
  INPUT section_name
  INPUT subject
  INPUT section
  INPUT room

  query = "INSERT INTO sections (section_name,
subject, section, room, user_id, created_at)
       VALUES ('" + section_name + "', '" + subject + "',
'" + section + "', '" + room + "', " + current_user_id + ",
CURRENT_TIMESTAMP)"

  IF ExecuteQuery(query) THEN
    DISPLAY "Class added successfully"
  ELSE
    DISPLAY "Error adding class"
  END IF

  CALL ManageClasses()
END FUNCTION
```

- **Attendance Management**

```
FUNCTION ManageAttendance()
  DISPLAY "Attendance Management"
  DISPLAY Options:
    1. View Attendance
    2. Mark Attendance
    3. Edit Attendance
    4. Back to Dashboard
  INPUT action
  SWITCH action
    CASE 1: CALL ViewAttendance()
    CASE 2: CALL MarkAttendance()
    CASE 3: CALL EditAttendance()
    CASE 4: CALL LoadDashboard()
  END SWITCH
END FUNCTION
```

```
FUNCTION MarkAttendance()
  sections = GetAllSections()
  DISPLAY "Select Section:"
  FOR EACH section IN sections DO
    DISPLAY section.section_id + ". " +
section.section_name
  END FOR
  INPUT section_id
  students = GetStudentsBySection(section_id)
  INPUT attendance_date
  FOR EACH student IN students DO
    DISPLAY student.first_name + " " +
student.last_name
    INPUT status // Present, Absent, Late
    query = "INSERT INTO attendance
(attendance_id, section_id, student_id,
attendance_date, status, created_at)
        VALUES (NULL, " + section_id + ", " +
student.student_id + ", '" + attendance_date + "', '" +
status + "', CURRENT_TIMESTAMP)"
    ExecuteQuery(query)
  END FOR
  DISPLAY "Attendance marked successfully"
  CALL ManageAttendance()
END FUNCTION
```

- **Grade Management**

```
FUNCTION ManageGrades()
  DISPLAY "Grade Management"
  DISPLAY Options:
    1. View Grades
    2. Add Grade
    3. Edit Grade
    4. Calculate Final Grades
    5. Back to Dashboard
  INPUT action
  SWITCH action
    CASE 1: CALL ViewGrades()
    CASE 2: CALL AddGrade()
    CASE 3: CALL EditGrade()
    CASE 4: CALL CalculateFinalGrades()
    CASE 5: CALL LoadDashboard()
  END SWITCH
END FUNCTION
```

```
FUNCTION AddGrade()
  INPUT student_id
  INPUT section_id
  INPUT midterm
  INPUT final
  query = "INSERT INTO grades (student_id,
section_id, midterm, final, created_at)
      VALUES (" + student_id + ", " + section_id + ", "
+ midterm + ", " + final + ", CURRENT_TIMESTAMP)"
  IF ExecuteQuery(query) THEN
    DISPLAY "Grade added successfully"
  ELSE
    DISPLAY "Error adding grade"
  END IF
  CALL ManageGrades()
END FUNCTION
```

- **Schedule Management**

```
FUNCTION ManageSchedule()
  DISPLAY "Schedule Management"
  DISPLAY Options:
    1. View Schedule
    2. Add Schedule
    3. Edit Schedule
    4. Back to Dashboard
  INPUT action
  SWITCH action
    CASE 1: CALL ViewSchedule()
    CASE 2: CALL AddSchedule()
    CASE 3: CALL EditSchedule()
    CASE 4: CALL LoadDashboard()
  END SWITCH
END FUNCTION
```

```
FUNCTION ViewSchedule()
  query = "SELECT * FROM schedules ORDER BY day,
time"
  schedules = ExecuteQuery(query)
  FOR EACH schedule IN schedules DO
    DISPLAY schedule.subject + " - " +
                                  schedule.day + " " +
  CALL ManageSchedule()      schedule.room
END FUNCTION
```

- **Resources Management**

```
FUNCTION ManageResources()
  DISPLAY "Resource Management"
  DISPLAY Options:
    1. View Resources
    2. Upload Resource
    3. Delete Resource
    4. Back to Dashboard
  INPUT action
  SWITCH action
    CASE 1: CALL ViewResources()
    CASE 2: CALL UploadResource()
    CASE 3: CALL DeleteResource()
    CASE 4: CALL LoadDashboard()
  END SWITCH
END FUNCTION
```

```
FUNCTION UploadResource()
  INPUT file_name
  INPUT subject
  INPUT file_path
  query = "INSERT INTO resources (user_id,
file_name, subject, file_path, file_size, uploaded_at)
        VALUES (" + current_user_id + ", '" + file_name
+ "', '" + subject + "', '" + file_path + "', " + file_size + ",
CURRENT_TIMESTAMP)"
  IF ExecuteQuery(query) THEN
    DISPLAY "Resource uploaded successfully"
  ELSE
    DISPLAY "Error uploading resource"
  END IF

  CALL ManageResources()
END FUNCTION
```

- **Logout**

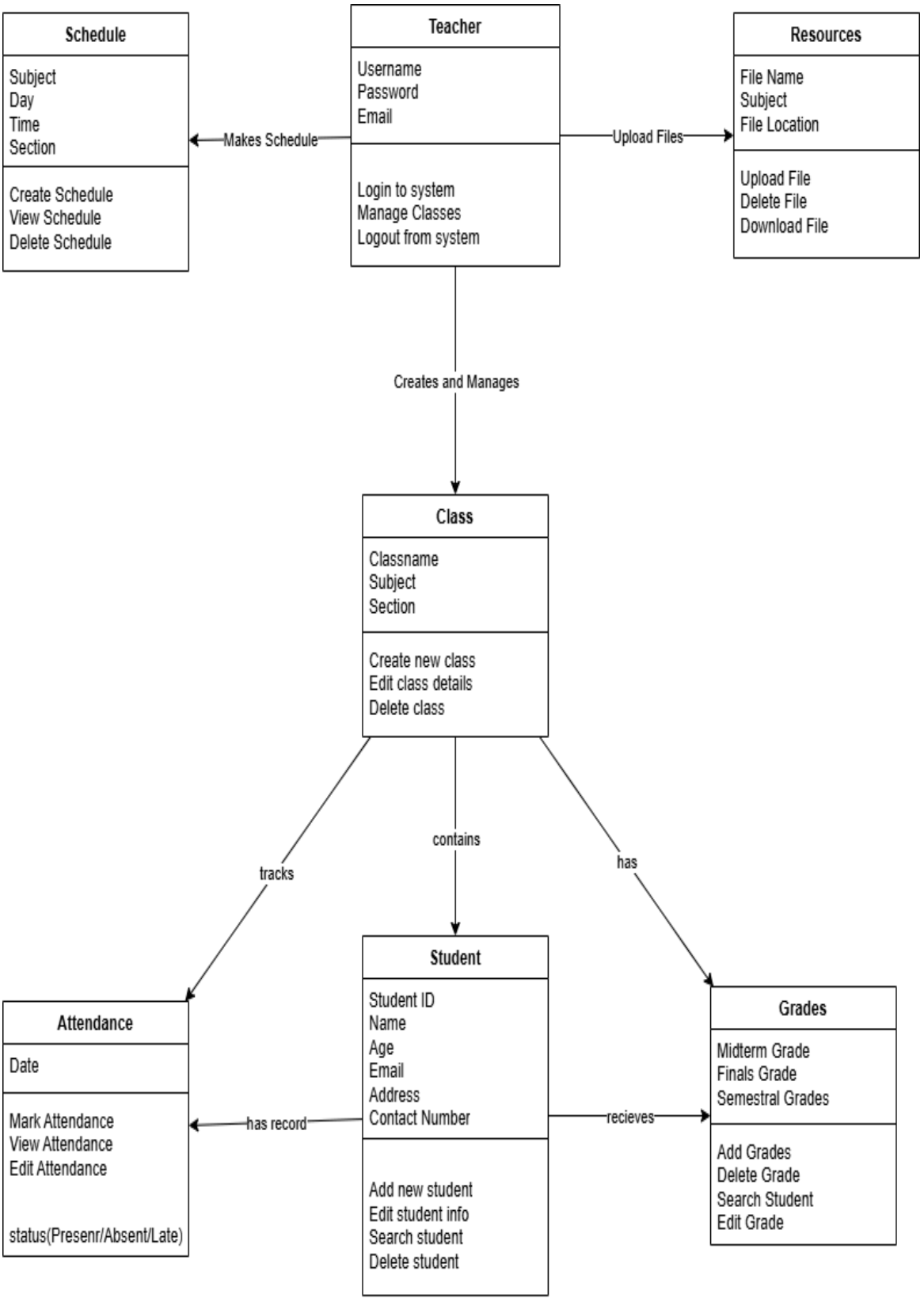
```
FUNCTION Logout()
  DISPLAY "Logging out..."
  CALL DisplayLoginScreen()
END FUNCTION
```

- **Helper Functions**

```
FUNCTION ExecuteQuery(query)
  // Execute SQL query and return result
  RETURN database.execute(query)
END FUNCTION

FUNCTION GetUserData(username)
  query = "SELECT * FROM users WHERE username
= '" + username + "'"
  RETURN ExecuteQuery(query)
END FUNCTION
```

```
FUNCTION GetAllSections()
  query = "SELECT * FROM sections"
  RETURN ExecuteQuery(query)
END FUNCTION

FUNCTION GetStudentsBySection(section_id)
  query = "SELECT * FROM students WHERE
section_id = " + section_id
  RETURN ExecuteQuery(query)
END FUNCTION

END PROGRAM
```

UML Diagram:


Schedule
Subject
Day
Time
Section
Create Schedule
View Schedule
Delete Schedule
Makes Schedule
Teacher
Username
Password
Email
Login to system
Manage Classes
Logout from system
Upload Files
Resources
File Name
Subject
File Location
Upload File
Delete File
Download File
Creates and Manages
Class
Classname
Subject
Section
Create new class
Edit class details
Delete class
tracks
contains
has
Attendance
Date
Mark Attendance
View Attendance
Edit Attendance
status(Presenr/Absent/Late)
has record
Student
Student ID
Name
Age
Email
Address
Contact Number
Add new student
Edit student info
Search student
Delete student
recieves
Grades
Midterm Grade
Finals Grade
Semestral Grades
Add Grades
Delete Grade
Search Student
Edit Grade

Sample Input and Output:

Input

The input refers to the data that the **professor** provides through the **ClassTrack** application.

- **User Input**:
    - **Login Information**: The professor inputs their **username** and **password** to log in to the system.
    - **Class Management**: The professor can input information for creating new classes, such as:
        - Subject (e.g., "CMSC 203", "ITEC 104")
        - Section (e.g., "BSCS 2B", "BSCS 2A")
    - **Student Management:** The professor can add, delete, and edit student information including:
        - Student ID
        - First Name and Last Name
        - Age
        - Email
        - Phone Number
        - Birthday
        - Address
    - **Attendance Records**: The professor marks attendance for students by selecting Present or Absent for each student on a specific date.

- **Grade Input**: Professors can enter or update student grades including:
  - Midterm Grade (1.00-5.00 scale)
  - Final Grade (1.00-5.00 scale)
  - Semestral Grade (automatically calculated)
- **Search Queries**: The professor can search for students, classes, or attendance records using the search bar in the **top bar**.
- **Schedule Management:** The professor can create class schedules by entering:
  - Subject
  - Section
  - Day (Monday-Saturday)
  - Time (e.g., "7:00-9:00")
  - Room (e.g., "101", "Room 301")
- **Resources Upload:** The professor can upload learning materials including:
  - Resource Name
  - Subject
  - File Selection (PDF, images, documents)
- **Search Queries:** The professor can search for students, classes, attendance records, or resources using search bars throughout the application.

- **System Input:**
    - **Student Data:** The system retrieves student information (names, attendance records, and grades) from the MySQL database.
    - **Class Data:** The system retrieves class/section information, schedules, and student enrollment from the database.
    - **Attendance Data:** The system retrieves historical attendance records for analysis and display.
    - **Grade Data:** The system retrieves midterm, final, and semestral grades for student performance tracking.
    - **Resource Data:** The system retrieves uploaded learning materials from the database and file storage.

Output

The output is the information displayed to the **professor** after they interact with the system.

- **User Output:**
    - **Dashboard:** After logging in, the professor sees:
        - Welcome message with their **username**
        - **Statistics** cards showing Total Classes, Total Students, Today's Classes, and Total Resources
        - **Outstanding Students** section displaying top 10 students ranked by semestral grades (1.00 = highest)
        - Student cards showing student names and their grades

- **My Classes Page:** Displays all active classes with:
  - Class cards showing subject and section
  - Student count per class
  - Options to View, Edit, or Archive classes
  - **"CREATE CLASS"** button for adding new classes
- **Class Detail View:** Shows comprehensive class information including:
  - Students Tab: List of all enrolled students with their details
  - Attendance Tab: All attendance records with date, present/absent counts
  - Grades Tab: Student grades (Midterm, Final, Semestral) with grade interpretation
- **Attendance Records: Professors can:**
  - View attendance by date with Present/Absent statistics
  - Create new attendance records
  - Edit or delete existing attendance records
  - View archived attendance
- **Grade Display: Shows student grades with:**
  - Midterm and Final grades
  - Semestral grade (average of Midterm and Final)
  - Grade interpretation (Excellent, Very Good, Good, Fair, Failed**:**
- **Schedule Page:** Displays:
  - Weekly schedule grid (Monday-Saturday, 7:00 AM - 5:00 PM)

        - Color-coded class blocks showing Subject, Section, Time, and Room
        - Today's Schedule panel showing current day's classes
    - **Resources Page:** Shows uploaded learning materials organized by:
        - Subject categories
        - Resource cards with file information
        - Options to View, Download, or Delete resources
    - **Archive Page:** Lists archived classes with options to:
        - Restore classes
        - Permanently delete classes
        - Select multiple classes for bulk deletion
    - **Search Results:** When searching, the system displays filtered results for students, classes, or resources based on the query**.**

- **System Output**:
    - **Database Updates:** The system automatically saves all changes to the MySQL database including:
        - New classes, students, attendance records, and grades
        - Schedule entries and resource uploads
        - Archive and restore operations
    - **Success/Error Messages:** The system provides feedback through message boxes:
        - Success confirmations (e.g., "Class created successfully")

        - Error notifications (e.g., "Student ID already exists")
        - Validation warnings (e.g., "Please fill in all required fields")
    - **Visual Indicators:** The system provides:
        - Color-coded class cards and schedule blocks
        - Grade interpretations with performance indicators
        - **Attendance statistics (Present/Absent counts)**

Example of Input and Output:

- **Input Example**:
    - The professor logs in with username: "**roxanne**" and password: "**roxanne**".
    - The professor creates a new class by entering:
        - Subject: "**CMSC 203**"
        - Section: "**BSCS 2B**"
    - The professor adds a student with:
        - Student ID: "**2022-0001**"
        - Name: "**Marciano Dela Cruz**"
        - Section: "**BSCS 2B**"
    - The professor marks attendance for December 9, 2025:
        - 22 students marked as **Present**

        - 4 students marked as **Absent**
    - The professor enters grades for a student:
        - Midterm: **1.75**
        - Final: 1**.50**
- **Output Example**:
    - After logging in, the professor is directed to the Dashboard, which displays:
        - Welcome message: "Welcome back, Roxanne!"
        - Total Classes: 2
        - Total Students: 26
        - Today's Classes: 1
        - Outstanding Students list showing top performers by grade
    - The system confirms: "Class created successfully!" and the new class "CMSC 203 - BSCS 2B" appears in My Classes.
    - The system confirms: "Student added successfully!" and Juan Dela Cruz appears in the CMSC 203 student list.
    - The attendance record for December 9, 2025 is saved showing:
        - Total: 26 students
        - Present: 22
        - Absent: 4
    - The system calculates and displays:
        - Semestral Grade: 1.63 (average of 1.75 and 1.50)
        - Interpretation: "Satisfactory"

System Design

- **Data Structures**:
    - **QStackedWidget**: This widget is used to manage multiple pages within the application. It allows different views (such as the dashboard, classes, attendance, and settings) to be stacked on top of one another, with only one visible at any given time.

    - **Layouts (QVBoxLayout, QHBoxLayout)**: These layouts help organize the widgets (buttons, labels, etc.) in a clear and structured way, both horizontally and vertically. They ensure that the content is displayed in an orderly fashion.

    - **QPushButton**: This widget is used to create interactive buttons for navigation. Each button corresponds to a page or a section in the application.

    - **QLineEdit**: Used for text input, such as entering the username and password during the login process.

    - **QFrame**: Frames are used to group and organize sections of the interface, such as the sidebar, top bar, and content area.

- **Algorithms**:

    - The primary algorithm used in this application is the **signal-slot mechanism** provided by PyQt5. This approach allows the application to respond to user actions such as clicking buttons, entering text, and switching between pages. When an event occurs, a corresponding action is triggered, such as switching pages or updating displayed data.

    - **Page Switching Algorithm**: The application uses a straightforward approach to switch between different sections of the UI, utilizing a **QStackedWidget** to handle transitions. The algorithm checks which button is clicked and displays the corresponding page by changing the index of the **QStackedWidget**.

Justifications:

- **QStackedWidget** was chosen for managing multiple pages because it allows seamless transitions between different views without needing to open multiple windows, improving user experience and efficiency.

- **Layouts** like **QVBoxLayout** and **QHBoxLayout** are used to organize UI components in a flexible and responsive way, ensuring that the application looks good across different screen sizes.

Object-Oriented Programming Concepts Applied

In the development of **ClassTrack**, we used several **Object-Oriented Programming (OOP)** principles to organize the code and make the system easy to manage, extend, and update. These principles include **Encapsulation**, **Inheritance**, **Polymorphism**, and **Abstraction**.

---

1. Encapsulation

- **What it means**: Encapsulation is about keeping data (like student information) and the methods that work on it (like marking attendance) together in one unit (class). It hides the details and only shows the necessary parts to the user.

- **How it's used in ClassTrack**:

    - Each part of the system, such as the **Dashboard** or **My Classes Panel**, is built as a separate class, and each class handles its own data and actions. This makes the code more organized and easier to manage.

2. Inheritance

- **What it means**: Inheritance allows a class to inherit properties and methods from another class. This lets us reuse code and create new features without writing everything from scratch.
- **How it's used in ClassTrack**:
    - Classes like **DashboardWindow** and inherit from PyQt5's basic classes, so they get basic window features and can focus on custom functionality like managing classes or attendance.

3. Polymorphism

- **What it means**: Polymorphism means that different classes can have methods with the same name, but each method can do different things. This allows flexibility in how the program responds to actions.
- **How it's used in ClassTrack**:
    - For example, when a button is clicked, it can trigger different actions depending on which page (Dashboard, Attendance, etc.) the user is on. This makes the system more adaptable.

4. Abstraction

- **What it means**: Abstraction hides the complex details and only shows what is necessary. In ClassTrack, the user interacts with simple buttons and options, while the complex processes (like database updates or data calculations) happen in the background.

- **How it's used in ClassTrack**:

    - Teachers can interact with a simple, clean interface for tasks like marking attendance, while the system handles the complicated work of storing data and managing class records behind the scenes.

Conclusion:

The **ClassTrack** application provides an innovative solution to address the increasing complexity of managing classroom activities. Designed specifically for educators, it integrates essential tools for class scheduling, attendance tracking, and student performance monitoring into a single, user-friendly platform. By streamlining these administrative tasks, **ClassTrack** allows professors to efficiently manage their classes while saving valuable time and effort.

The system's intuitive interface, built using the **PyQt5** framework, ensures ease of use with minimal training required for professors. It provides a seamless experience, enabling educators to quickly access and interact with critical data such as attendance records and student progress. Additionally, the ability to generate charts and graphs based on real-time data gives professors a powerful tool to visualize trends and identify areas where students may need additional support.

One of the most significant advancements in **ClassTrack** is its integration with a **database**, allowing for real-time data management and efficient storage. This integration ensures that the system can dynamically update and store class data,

making it more reliable and responsive. The collaboration between the **CMSC** and **ITEC** courses has been crucial in this development, providing the necessary foundation for scalability. As the number of users and data grows, **ClassTrack** will be able to handle larger datasets effectively, adapting to the evolving needs of educational institutions.

Recommendations:

While **ClassTrack** already provides a comprehensive solution for managing classroom activities, there are several areas where future improvements could enhance its functionality and user experience for professors.

One key recommendation is to **expand the database integration** to support **cloud synchronization**, enabling professors to access and manage their data across multiple devices. This would improve flexibility, especially for instructors who may need to work from different locations. Cloud integration would also ensure that data is securely backed up and accessible in real-time, improving the overall reliability of the system.

Another suggestion is to improve the **charting and data visualization** features**.** Right now, the application offers basic charts and graphs, but adding more interactive features, like detailed charts or real-time tracking of student performance, could help professors better understand student progress and attendance. These improvements would allow professors to make more informed decisions to enhance student engagement and learning.

Additionally, the platform could be extended to support **mobile devices for professors**, allowing them to access and update class schedules, attendance records, and grades on smartphones and tablets. This would increase accessibility and convenience for professors, enabling them to stay connected and manage their classes efficiently, regardless of location.

Finally, incorporating **real-time notifications** for important updates, such as class schedule changes, attendance alerts, or grade updates, would keep professors informed and ensure they can quickly address any changes or issues within their courses.